

Trabalho Prático — API com Filas de Prioridade e Heaps

Curso: Sistemas de Informação | **Disciplina:** Códigos de Alta Performance

Professor: Ricardo Fulgencio Alves

Valor: 2,0 pontos | **Entrega:** Repositório Git com projeto funcionando, documentação e apresentação em sala

1. Contextualização

Neste trabalho, cada equipe deverá desenvolver uma API REST funcional para resolver um cenário realista em que os itens de uma fila não devem ser atendidos simplesmente por ordem de chegada. Em muitos sistemas reais, como pronto-socorro, filas bancárias, despacho de entregas, chamados de manutenção ou processamento de pagamentos, a próxima solicitação a ser atendida depende de critérios como urgência, risco, prazo, impacto, valor, gravidade ou tempo de espera.

A atividade tem como objetivo conectar os conceitos estudados em aula sobre filas de prioridade e Heaps com uma implementação prática de software. A equipe deverá desenvolver uma API com persistência em banco de dados, documentação Swagger/OpenAPI, organização arquitetural inspirada em Clean Architecture e DDD, aplicação de princípios SOLID e explicação clara da estrutura de dados utilizada.

Uma fila comum do tipo **FIFO** atende primeiro quem chegou primeiro. Uma fila de prioridade, por outro lado, atende primeiro o item mais relevante segundo uma regra definida pelo domínio do problema.

A documentação da API deverá seguir o padrão OpenAPI/Swagger, pois a especificação OpenAPI define uma descrição padronizada e independente de linguagem para APIs HTTP, permitindo que humanos e ferramentas compreendam as capacidades de um serviço sem depender do código-fonte. O banco de dados deverá ser executado via Docker Compose, ferramenta que permite definir e executar aplicações com múltiplos serviços a partir de um arquivo YAML.

2. Objetivos de Aprendizagem

Ao final do trabalho, espera-se que a equipe seja capaz de projetar, implementar e explicar uma API baseada em uma fila de prioridade. O foco da atividade não é apenas construir endpoints CRUD, mas demonstrar domínio sobre a regra de negócio, a estrutura de dados utilizada e a motivação técnica por trás da solução.

Objetivo	O que a equipe deve demonstrar
Compreender filas de prioridade	Explicar por que a ordem de atendimento depende de prioridade e não apenas de chegada.
Aplicar Heaps	Explicar onde o conceito de Heap se encaixa na solução e por que ele é adequado.
Modelar uma API REST	Criar endpoints coerentes, documentados e testáveis.
Persistir dados	Conectar a API a um banco de dados executado via Docker Compose.
Aplicar boas práticas	Organizar o código com separação de responsabilidades, SOLID e arquitetura em camadas.
Comunicar decisões técnicas	Apresentar cenário, arquitetura, regra de prioridade, endpoints e demonstração funcional.

3. Requisitos Obrigatórios do Projeto

Cada equipe deverá entregar um projeto funcional em um repositório Git. O projeto deve conter código-fonte, documentação, configuração de banco de dados, instruções de execução e demonstração dos endpoints. A equipe poderá escolher a linguagem e o framework, desde que consiga atender aos requisitos técnicos e explicar as decisões adotadas.

Requisito obrigatório	Descrição
Repositório Git	O projeto deve estar em um repositório Git com README completo e histórico de commits.
API funcionando	A aplicação deve executar corretamente e expor os endpoints obrigatórios.
Banco de dados	A API deve possuir conexão real com banco de dados.
Docker Compose	O banco de dados deve estar configurado em docker-compose.yml.
Swagger/OpenAPI	A API deve possuir documentação interativa dos endpoints, parâmetros, payloads e respostas.
Fila de prioridade	O processo principal deve utilizar uma lógica explícita de prioridade.
Regra de prioridade explícita	A prioridade deve ser calculada ou derivada dos dados do domínio, não apenas informada manualmente.
Tratamento de empate	A equipe deve definir e implementar uma regra para itens com mesma prioridade.
Exclusão lógica	O DELETE não deve remover fisicamente o registro, apenas alterar seu status.
Estrutura de dados adequada	A equipe deve justificar a estrutura utilizada para organizar a fila.
Aplicação de Heap	A equipe deve explicar onde o Heap se enquadra e por que é útil para o problema.
Princípios SOLID	O código deve demonstrar separação de responsabilidades, baixo acoplamento e organização.
Apresentação da arquitetura	A equipe deve explicar a estrutura do projeto durante a apresentação.

4. Entregáveis

A entrega deverá permitir que o professor clone o repositório, suba o banco de dados, execute a aplicação e teste os endpoints. O README será considerado parte da avaliação, pois representa a documentação mínima de um projeto acadêmico-profissional.

Entregável	Descrição esperada
Link do repositório Git	Repositório com o projeto completo e acessível para avaliação.
Código-fonte da API	Projeto organizado, compilável/executável e com endpoints implementados.
docker-compose.yml	Arquivo para subir o banco de dados necessário à aplicação.
README.md	Instruções de execução, descrição do cenário, regra de prioridade, endpoints e exemplos.
Swagger/OpenAPI	Documentação interativa acessível ao executar a aplicação.
Demonstração em sala	Apresentação do cenário, arquitetura, prioridade, Heap e endpoints funcionando.
Massa de dados ou exemplos	Exemplos de payloads JSON para facilitar os testes.
Testes unitários da prioridade	Recomendados para validar regra de prioridade e tratamento de empate.

5. Endpoints Obrigatórios

Cada projeto deverá implementar exatamente o conjunto mínimo de operações abaixo. O nome do recurso deve ser adequado ao cenário escolhido. Por exemplo, no pronto-socorro o recurso pode ser `/pacientes`; em entregas, `/entregas`; em pagamentos, `/transacoes`.

O endpoint `DELETE /recurso/{id}` deve ser implementado como exclusão lógica. Isso significa que o registro não deve ser apagado fisicamente do banco. O sistema deve apenas alterar o status do item para algo como `EXCLUIDO`, `INATIVO`, `CANCELADO` ou `REMOVIDO`. Após essa alteração, o item não deve aparecer nas consultas comuns feitas por `GET`.

Método	Endpoint obrigatório	Descrição
POST	<code>/recurso</code>	Cadastrar um novo item na fila.
GET	<code>/recurso/{id}</code>	Buscar um item específico pelo identificador.
GET	<code>/recurso?page=1&size=10</code>	Listar itens ativos de forma paginada, com 10 itens por página.
GET	<code>/recurso/buscar?cpf=...</code> ou <code>/recurso/buscar?descricao=...</code>	Buscar um item por CPF, quando aplicável, ou por descrição.
PUT	<code>/recurso/{id}</code>	Atualizar dados do item e recalcular a prioridade, se necessário.
DELETE	<code>/recurso/{id}</code>	Realizar exclusão lógica, alterando apenas o status no banco de dados.

6. Endpoints Extras Recomendados

Os endpoints abaixo não substituem os obrigatórios, mas são recomendados porque ajudam a demonstrar melhor o uso de fila de prioridade no domínio escolhido.

Método	Endpoint recomendado	Descrição
GET	/recurso/proximo	Consulta o próximo item de maior prioridade sem alterar seu status.
POST	/recurso/proximo/atender	Seleciona o próximo item prioritário e altera seu status para EM_ATENDIMENTO.
PATCH	/recurso/{id}/status	Atualiza apenas o status do item.
GET	/recurso/estatisticas	Retorna indicadores simples, como total aguardando, em atendimento e concluídos.

7. Estrutura Desejada da Solução

A solução deve ser organizada de forma inspirada em Clean Architecture e Domain-Driven Design — DDD. A estrutura abaixo é uma sugestão. Ela pode ser adaptada conforme a linguagem, framework ou padrão escolhido pela equipe, desde que a separação de responsabilidades fique evidente.

```
NomeDaApi/  
├── src/  
│   ├── NomeDaApi.Api/  
│   │   ├── Controllers/  
│   │   ├── DTOs/  
│   │   ├── Requests/  
│   │   ├── Responses/  
│   │   ├── Middlewares/  
│   │   ├── Filters/  
│   │   ├── Configurations/  
│   │   └── Program.cs ou main  
│   ├── NomeDaApi.Application/  
│   │   ├── UseCases/  
│   │   ├── Services/  
│   │   ├── Interfaces/  
│   │   ├── Validators/  
│   │   └── Mappings/  
│   └── NomeDaApi.Domain/  
│       ├── Entities/  
│       ├── ValueObjects/  
│       ├── Enums/  
│       ├── Services/  
│       ├── Interfaces/  
│       └── PriorityRules/
```

```
├── NomeDaApi.Infrastructure/
│   ├── Persistence/
│   ├── Repositories/
│   ├── Migrations/
│   ├── Configurations/
│   └── ExternalServices/
├── tests/
│   └── NomeDaApi.UnitTests/
├── docker-compose.yml
├── README.md
└── .gitignore
```

8. Responsabilidade Esperada de Cada Camada

A equipe pode implementar a solução em um único projeto de API separado por pastas ou em múltiplos projetos (como uma solução clássica do .NET). O mais importante é que as responsabilidades estejam bem definidas:

- **Camada de Apresentação (API):** Deve conter apenas a recepção das requisições, validações simples de formato de dados, mapeamento para DTOs e retorno das respostas HTTP. Não deve conter regras de negócio e nem acesso direto ao banco.
- **Camada de Aplicação:** Coordena a execução das ações. Ela recebe os dados da API, busca informações através das interfaces dos repositórios, chama as regras do domínio e salva as alterações.
- **Camada de Domínio:** Onde ficam o coração do software. Contém as entidades, enums e a lógica de cálculo de prioridade. Essa camada não deve depender de nenhuma outra camada ou framework externo (como ORMs ou drivers de banco).
- **Camada de Infraestrutura:** Implementa o acesso ao banco de dados (reais repositórios), migrações, mapeamento de tabelas e qualquer comunicação externa.

9. Estrutura Mínima Esperada no Banco de Dados

Cada cenário terá campos específicos, mas todos devem possuir uma entidade principal associada à fila. Abaixo estão os campos mínimos necessários para que o projeto consiga rodar e atender aos critérios de avaliação.

Campo sugerido	Finalidade
id	Identificador único do item.
cpf	Identificação da pessoa, quando o cenário envolver cliente, paciente, aluno ou usuário.
descricao	Descrição do problema, pedido, chamado, solicitação ou ocorrência.
prioridade	Valor calculado ou derivado que define a posição relativa na fila.
status	Estado atual do item, como AGUARDANDO, EM_ATENDIMENTO, CONCLUIDO ou EXCLUIDO.
dataCriacao	Data e hora de entrada na fila.
dataAtualizacao	Data e hora da última alteração.
dataExclusao	Data e hora da exclusão lógica, se aplicável.
ativo ou excluido	Campo auxiliar opcional para facilitar filtros de exclusão lógica.

A equipe deve criar campos adicionais conforme o domínio. No pronto-socorro, por exemplo, podem existir campos como sintomas e idade. Em entregas, peso e prazo limite.

10. Sobre a Regra de Prioridade

A prioridade deve ser explícita, justificável e coerente com o cenário. Ela não deve ser apenas um número digitado manualmente pelo usuário no Swagger. O usuário deve cadastrar as propriedades do recurso, e o sistema deve calcular de forma automatizada o valor da prioridade com base em uma fórmula ou tabela de decisão de negócio.

Também é obrigatório tratar empates. Se dois itens tiverem a mesma prioridade, a equipe deve aplicar um critério secundário de ordenação para definir quem deve ser atendido primeiro. Recomenda-se utilizar a data e hora de criação (`dataCriacao`) para que o item mais antigo (que está esperando há mais tempo) tenha preferência.

Cenário	Exemplos de fatores de prioridade
Pronto-socorro	Gravidade, classificação de risco, idade, sintomas e tempo de espera.
Entregas urbanas	Prazo prometido, distância, risco de atraso e valor do pedido.
Help desk de TI	Impacto no negócio, urgência, setor afetado e número de usuários impactados.
Manutenção predial	Risco à segurança, impacto operacional, custo de parada e tempo de abertura.
Pagamentos	Valor da transação, risco antifraude, horário limite e tipo de cliente.

11. Onde o Heap se Encaixa

A equipe deve explicar que a fila de prioridade é o conceito abstrato, enquanto o Heap é uma estrutura de dados concreta altamente eficiente para implementar esse conceito. O Heap garante que o elemento de maior prioridade esteja sempre na raiz (tempo constante $O(1)$ para consulta), e que as operações de inserção e remoção ocorram em tempo logarítmico $O(\log n)$, o que é ideal para sistemas de alta performance.

Explicação Esperada na Apresentação: “Utilizamos uma fila de prioridade porque o próximo item a ser atendido não depende apenas de quem chegou primeiro, mas sim da urgência do negócio. E explicamos que o Heap apoia essa solução por permitir gerenciar a árvore de prioridades com complexidade $O(\log n)$ nas operações de inserção e remoção do topo.”

Mesmo que a equipe utilize uma estrutura pronta da linguagem (como `PriorityQueue` no .NET, `heapq` no Python, ou `PriorityBlockingQueue` no Java), os alunos devem saber explicar como a estrutura funciona por baixo dos panos (as propriedades do Heap, como o rearranjo ocorre na subida e na descida).

12. Cenários Disponíveis

Cada equipe deverá escolher ou receber um dos cenários abaixo. O nome sugerido da API pode ser utilizado para batizar o repositório e o projeto.

Nº	Cenário	Nome sugerido da API	Recurso principal	Endpoint base sugerido
1	Fila de atendimento bancário	BankPriorityQueueApi	Atendimento bancário	/atendimentos-bancarios
2	Pronto-socorro hospitalar	EmergencyTriageApi	Paciente	/pacientes
3	Fila de suporte acadêmico ou monitoria	AcademicSupportQueueApi	Solicitação acadêmica	/solicitacoes-academicas
4	Help desk de TI interno	InternalHelpDeskApi	Chamado de TI	/chamados-ti
5	Central de entregas urbanas	UrbanDeliveryPriorityApi	Entrega	/entregas
6	Sistema de impressão corporativa	CorporatePrintQueueApi	Trabalho de impressão	/impressoes
7	Atendimento em restaurante ou praça de alimentação	RestaurantOrderQueueApi	Pedido	/pedidos
8	Gestão de chamados de manutenção predial	BuildingMaintenanceApi	Ordem de serviço	/ordens-servico
9	Despacho de ambulâncias ou viaturas	EmergencyDispatchApi	Ocorrência	/ocorrencias
10	Processamento de pagamentos	PaymentProcessingQueueApi	Transação	/transacoes

12.1 Fila de Atendimento Bancário — BankPriorityQueueApi

Seu time foi designado a desenvolver uma API para atender ao cenário de uma agência bancária que precisa gerenciar o fluxo de atendimento de clientes. A agência atende tanto por ordem de chegada quanto por critérios legais de prioridade e segmentação comercial.

Neste cenário, a equipe deve explicar como clientes idosos, pessoas com deficiência, clientes premium e clientes comuns são distribuídos na fila de atendimento, garantindo o cumprimento das leis de prioridade sem travar completamente o atendimento dos demais clientes.

Endpoint	Aplicação no cenário
POST /atendimentos-bancarios	Cadastra um cliente na fila de atendimento.
GET /atendimentos-bancarios/{id}	Consulta um atendimento específico.
GET /atendimentos-bancarios?page=1&size=10	Lista atendimentos ativos de forma paginada.
GET /atendimentos-bancarios/buscar?cpf=...	Busca cliente pelo CPF.
PUT /atendimentos-bancarios/{id}	Atualiza serviço, status ou dados que impactem a prioridade.
DELETE /atendimentos-bancarios/{id}	Realiza exclusão lógica do atendimento.

12.2 Pronto-Socorro Hospitalar — EmergencyTriageApi

Seu time foi designado a desenvolver uma API para atender ao cenário de um pronto-socorro hospitalar, onde a triagem dos pacientes define a ordem imediata de atendimento médico. O sistema baseia-se em protocolos de Manchester adaptados.

Neste cenário, a equipe deve demonstrar que a fila não pode ser puramente cronológica, pois pacientes com risco de morte iminente (emergência) devem furar a fila de pacientes com sintomas leves, justificando o uso de pesos numéricos para cada cor de triagem combinada com o tempo de espera.

Endpoint	Aplicação no cenário
POST /pacientes	Cadastra paciente na fila de atendimento.
GET /pacientes/{id}	Consulta paciente pelo identificador.
GET /pacientes?page=1&size=10	Lista pacientes ativos de forma paginada.
GET /pacientes/buscar?cpf=...	Busca paciente pelo CPF.
PUT /pacientes/{id}	Atualiza sintomas, classificação de risco ou status.
DELETE /pacientes/{id}	Realiza exclusão lógica do paciente.

12.3 Suporte Acadêmico ou Monitoria — AcademicSupportQueueApi

Seu time foi designado a desenvolver uma API para atender ao cenário de uma central de suporte acadêmico ou plantão de monitoria de uma universidade, onde alunos solicitam auxílio com dúvidas e trabalhos práticos de disciplinas complexas.

Neste cenário, a equipe deve explicar como equilibrar urgência acadêmica (proximidade da data de entrega do trabalho ou prova) e justiça entre alunos que solicitam ajuda, impedindo que um único aluno monopolize o monitor por todo o período.

Endpoint	Aplicação no cenário
POST /solicitacoes-academicas	Cadastra uma solicitação de suporte acadêmico.
GET /solicitacoes-academicas/{id}	Consulta solicitação específica.
GET /solicitacoes-academicas?page=1&size=10	Lista solicitações ativas de forma paginada.
GET /solicitacoes-academicas/buscar?descricao=...	Busca solicitação por descrição.
PUT /solicitacoes-academicas/{id}	Atualiza disciplina, prazo, descrição ou prioridade calculada.
DELETE /solicitacoes-academicas/{id}	Realiza exclusão lógica da solicitação.

12.4 Help Desk de TI Interno — InternalHelpDeskApi

Seu time foi designado a desenvolver uma API para atender ao cenário de um help desk interno de TI, que recebe chamados de funcionários de diversos setores da empresa sobre problemas em equipamentos, softwares, acessos ou infraestrutura de rede.

Neste cenário, a equipe deve justificar por que um problema que afeta toda a empresa (ex: queda do servidor principal) deve ter prioridade absoluta sobre um chamado individual (ex: troca de mouse), utilizando fatores como impacto corporativo e cargo do solicitante.

Endpoint	Aplicação no cenário
POST /chamados-ti	Cadastra um chamado de TI.
GET /chamados-ti/{id}	Consulta um chamado específico.
GET /chamados-ti?page=1&size=10	Lista chamados ativos de forma paginada.
GET /chamados-ti/buscar?descricao=...	Busca chamado por descrição.
PUT /chamados-ti/{id}	Atualiza impacto, urgência, setor ou status.
DELETE /chamados-ti/{id}	Realiza exclusão lógica do chamado.

12.5 Central de Entregas Urbanas — UrbanDeliveryPriorityApi

Seu time foi designado a desenvolver uma API para atender ao cenário de uma central de entregas rápidas na cidade, responsável por despachar motoboys para entregar encomendas de e-commerce, documentos urgentes ou refeições.

Neste cenário, a equipe deve demonstrar como a fila de prioridade ajuda a reduzir atrasos e melhorar a eficiência operacional, priorizando pacotes com prazos de entrega quase expirados ou perecíveis sobre mercadorias sem urgência de prazo.

Endpoint	Aplicação no cenário
POST /entregas	Cadastra uma solicitação de entrega.
GET /entregas/{id}	Consulta uma entrega específica.
GET /entregas?page=1&size=10	Lista entregas ativas de forma paginada.
GET /entregas/buscar?descricao=...	Busca entrega por descrição, destinatário ou referência.
PUT /entregas/{id}	Atualiza endereço, prazo, status ou dados de prioridade.
DELETE /entregas/{id}	Realiza exclusão lógica da entrega.

12.6 Sistema de Impressão Corporativa — CorporatePrintQueueApi

Seu time foi designado a desenvolver uma API para atender ao cenário de uma empresa que possui uma fila centralizada de impressão compartilhada por diversos setores. O volume de impressões é alto e as impressoras de grande porte processam os trabalhos em lote.

Neste cenário, a equipe deve explicar como equilibrar documentos curtos e urgentes (ex: contrato para assinatura imediata) com trabalhos longos (ex: relatórios anuais de 500 páginas), evitando o fenômeno de starvation (onde impressões pequenas nunca rodam porque há arquivos gigantes sempre na frente).

Endpoint	Aplicação no cenário
POST /impressoes	Cadastra um trabalho de impressão.
GET /impressoes/{id}	Consulta trabalho de impressão específico.
GET /impressoes?page=1&size=10	Lista impressões ativas de forma paginada.
GET /impressoes/buscar?descricao=...	Busca impressão por descrição do documento.
PUT /impressoes/{id}	Atualiza páginas, setor, urgência ou status.
DELETE /impressoes/{id}	Realiza exclusão lógica do trabalho de impressão.

12.7 Atendimento em Restaurante ou Praça de Alimentação — RestaurantOrderQueueApi

Seu time foi designado a desenvolver uma API para atender ao cenário de um restaurante, lanchonete ou praça de alimentação, onde os pedidos feitos pelos clientes nos totens ou mesas são encaminhados para a tela da cozinha.

Neste cenário, a equipe deve explicar como a cozinha pode priorizar pedidos atrasados ou de preparo mais rápido para otimizar o giro das mesas, além de gerenciar pedidos de clientes prioritários ou de plataformas de delivery com tempo máximo estipulado.

Endpoint	Aplicação no cenário
POST /pedidos	Cadastra um novo pedido na fila da cozinha.
GET /pedidos/{id}	Consulta um pedido específico.
GET /pedidos?page=1&size=10	Lista pedidos ativos de forma paginada.
GET /pedidos/buscar?descricao=...	Busca pedido por descrição ou itens.
PUT /pedidos/{id}	Atualiza itens, status, modalidade ou prioridade calculada.
DELETE /pedidos/{id}	Realiza exclusão lógica do pedido.

12.8 Gestão de Chamados de Manutenção Predial — BuildingMaintenanceApi

Seu time foi designado a desenvolver uma API para atender ao cenário de manutenção predial, em que o administrador de um condomínio residencial ou empresarial recebe relatos de defeitos e precisa despachar a equipe técnica.

Neste cenário, a equipe deve justificar por que uma falha elétrica crítica ou vazamento grave em área comum deve ser resolvido antes de uma pintura estética de corredor, associando níveis de gravidade técnica aos chamados cadastrados.

Endpoint	Aplicação no cenário
POST /ordens-servico	Cadastra uma ordem de manutenção.
GET /ordens-servico/{id}	Consulta uma ordem específica.
GET /ordens-servico?page=1&size=10	Lista ordens ativas de forma paginada.
GET /ordens-servico/buscar?descricao=...	Busca ordem por descrição.
PUT /ordens-servico/{id}	Atualiza risco, local, responsável, status ou prioridade.
DELETE /ordens-servico/{id}	Realiza exclusão lógica da ordem de serviço.

12.9 Despacho de Ambulâncias ou Viaturas — EmergencyDispatchApi

Seu time foi designado a desenvolver uma API para atender ao cenário de despacho de ambulâncias, viaturas policiais ou carros de bombeiro a partir de uma central de atendimento telefônico de emergências.

Neste cenário, a equipe deve explicar como a prioridade pode ser recalculada quando novas ocorrências mais graves chegam ao sistema, exigindo que a viatura disponível seja direcionada para o evento de maior risco de vida.

Endpoint	Aplicação no cenário
POST /ocorrencias	Cadastra uma nova ocorrência.
GET /ocorrencias/{id}	Consulta uma ocorrência específica.
GET /ocorrencias?page=1&size=10	Lista ocorrências ativas de forma paginada.
GET /ocorrencias/buscar?descricao=...	Busca ocorrência por descrição.
PUT /ocorrencias/{id}	Atualiza gravidade, local, status ou prioridade.
DELETE /ocorrencias/{id}	Realiza exclusão lógica da ocorrência.

12.10 Processamento de Pagamentos — PaymentProcessingQueueApi

Seu time foi designado a desenvolver uma API para atender ao cenário de uma fila de processamento de transações financeiras, liquidações de boletos, transferências PIX ou remessas internacionais de um banco ou fintech.

Neste cenário, a equipe deve explicar como transações críticas (de alto valor ou com regras rígidas de horário limite — cutoff time) podem ser priorizadas na fila de execução para evitar multas contratuais e garantir liquidez ao sistema financeiro.

Endpoint	Aplicação no cenário
POST /transacoes	Cadastra uma transação na fila de processamento.
GET /transacoes/{id}	Consulta uma transação específica.
GET /transacoes?page=1&size=10	Lista transações ativas de forma paginada.
GET /transacoes/buscar?descricao=...	Busca transação por descrição ou referência.
PUT /transacoes/{id}	Atualiza valor, risco, status ou dados de prioridade.
DELETE /transacoes/{id}	Realiza exclusão lógica da transação.

13. Recomendações Técnicas

Além dos requisitos obrigatórios, as recomendações abaixo ajudam a elevar a qualidade do projeto e facilitam o processo de desenvolvimento e avaliação.

Recomendação	Descrição
README completo	Incluir cenário, arquitetura, endpoints, regra de prioridade, instruções de execução e exemplos de payload.
Exemplos de payload JSON	Demonstrar claramente como cadastrar, atualizar, buscar e excluir itens.
DTOs ou schemas	Separar objetos de entrada e saída das entidades internas do domínio.
Validação de entrada	Validar campos obrigatórios, formatos, status permitidos e valores numéricos.
Tratamento de erros	Retornar respostas adequadas para item não encontrado, dados inválidos e operações proibidas.
Testes unitários da prioridade	Validar itens com prioridades diferentes, empate e exclusão lógica.
Massa inicial de dados	Incluir seed ou instruções para cadastrar exemplos rapidamente.
Logs simples	Registrar operações importantes, como cadastro, atendimento e exclusão lógica.

14. Testes Unitários Recomendados

Os testes unitários da regra de prioridade são recomendados porque a prioridade é o núcleo conceitual do trabalho. Garantir que a lógica calcula o peso correto e que o desempate funciona previne falhas graves na API.

Caso de teste recomendado	Resultado esperado
Itens com prioridades diferentes	O item de maior prioridade deve ser selecionado primeiro.
Itens com mesma prioridade	O critério de desempate deve ser aplicado corretamente.
Item excluído logicamente	O item não deve aparecer nas listagens comuns nem participar da fila.
Atualização que altera prioridade	A posição lógica do item na fila deve ser recalculada.
Fila vazia	O sistema deve retornar resposta adequada sem erro inesperado.

15. Critérios de Avaliação — 2,0 Pontos

O trabalho vale 2,0 pontos. A avaliação considerará tanto o funcionamento técnico quanto a capacidade da equipe de explicar e defender as escolhas de arquitetura e a estrutura de dados Heap.

Critério	Pontuação	O que será avaliado
API funcional e endpoints obrigatórios	0,35	Todos os 6 endpoints funcionando, com busca, paginação e atualização adequadas.
Banco de dados e Docker Compose	0,20	Banco configurado no docker-compose.yml e integração funcionando.
Swagger/OpenAPI	0,15	Documentação interativa clara e utilizável para testar os endpoints.
Regra de prioridade e tratamento de empate	0,30	Prioridade explícita, coerente com o domínio e com regra de desempate implementada.
Aplicação de Heap e estrutura de dados	0,25	Explicação técnica de onde o Heap se enquadra e por que foi utilizado.
Exclusão lógica	0,15	DELETE alterando status e itens excluídos fora das consultas comuns.
Arquitetura, SOLID e boas práticas	0,30	Separação de responsabilidades, organização do projeto, validações e legibilidade.
Apresentação e domínio técnico	0,20	Clareza na explicação, demonstração funcionando e respostas às perguntas.
README e documentação do projeto	0,10	Instruções de execução, exemplos de uso e descrição da solução.
Total	2,00	

16. Roteiro Sugerido para Apresentação

Cada equipe deverá apresentar o projeto de forma objetiva. A apresentação deve demonstrar que a equipe domina o código e compreende os conceitos teóricos envolvidos.

Etapa	O que apresentar
1. Cenário	Qual problema real a API resolve.
2. Modelo de dados	Entidade principal, campos relevantes, status e campos de prioridade.
3. Regra de prioridade	Como a prioridade é calculada e como empates são tratados.
4. Heap	Onde o Heap se encaixa e por que uma fila FIFO comum não seria suficiente.
5. Arquitetura	Estrutura do projeto, camadas, responsabilidades e princípios SOLID aplicados.
6. Banco e Docker	Como subir o banco e conectar a aplicação.
7. Swagger/OpenAPI	Demonstração da documentação interativa.
8. Endpoints	Execução dos principais endpoints obrigatórios.
9. Exclusão lógica	Demonstração de que o item excluído não aparece mais nas consultas comuns.
10. Conclusão	Principais aprendizados e dificuldades técnicas.

17. Perguntas Obrigatórias para a Apresentação

Durante a apresentação, cada equipe deverá responder às perguntas abaixo. Elas serão usadas para avaliar o entendimento individual dos integrantes sobre o trabalho realizado.

Pergunta	O que se espera avaliar
Qual problema real sua API resolve?	Compreensão do cenário escolhido.
Qual é a regra de prioridade utilizada?	Clareza da lógica de negócio.
Por que uma fila FIFO comum não resolveria bem esse problema?	Entendimento da diferença entre fila comum e fila de prioridade.
Como o tratamento de empate foi implementado?	Compreensão de justiça e previsibilidade na fila.
Onde o Heap entra na solução?	Relação entre estrutura de dados e implementação prática.
Como a exclusão lógica foi implementada?	Persistência, status e filtragem nas consultas.
Onde aparecem os princípios SOLID no projeto?	Capacidade de explicar organização, responsabilidades e dependências.
Como executar o projeto com banco via Docker Compose?	Reprodutibilidade da entrega.
Como acessar e testar a API via Swagger/OpenAPI?	Documentação e usabilidade da API.

18. Orientações Finais

A equipe deve evitar implementar uma API que seja apenas um cadastro simples. O ponto central do trabalho é a existência de uma fila de prioridade real, com uma regra de negócio compreensível, defensável e conectada ao cenário escolhido. A prioridade deve poder ser explicada tanto do ponto de vista do usuário quanto do ponto de vista técnico.

Também é importante lembrar que a arquitetura não precisa ser exageradamente complexa. O objetivo é demonstrar separação de responsabilidades. Controllers não devem conter regra de negócio importante; repositórios não devem decidir prioridade; e a regra de prioridade deve estar em um local que possa ser lido, testado e explicado.

Por fim, cada equipe deve preparar uma demonstração curta, mas completa. Recomenda-se cadastrar pelo menos três itens com prioridades diferentes, demonstrar a listagem paginada, buscar um item, atualizar um dado que altere sua prioridade, realizar uma exclusão lógica e mostrar que o item excluído não aparece mais nas consultas comuns.